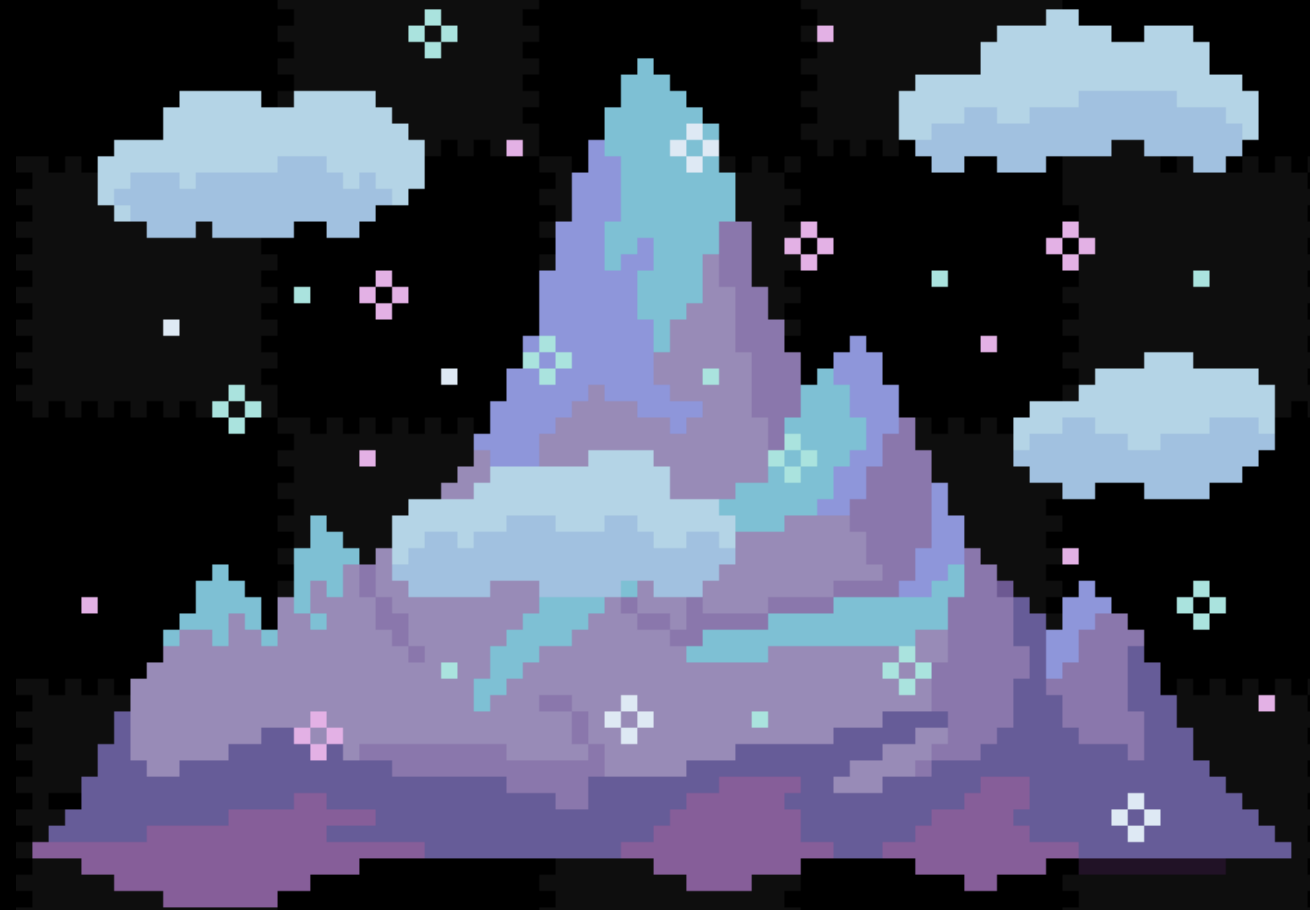


Reinforcement Learning & Behavioral Cloning for Celeste Classic

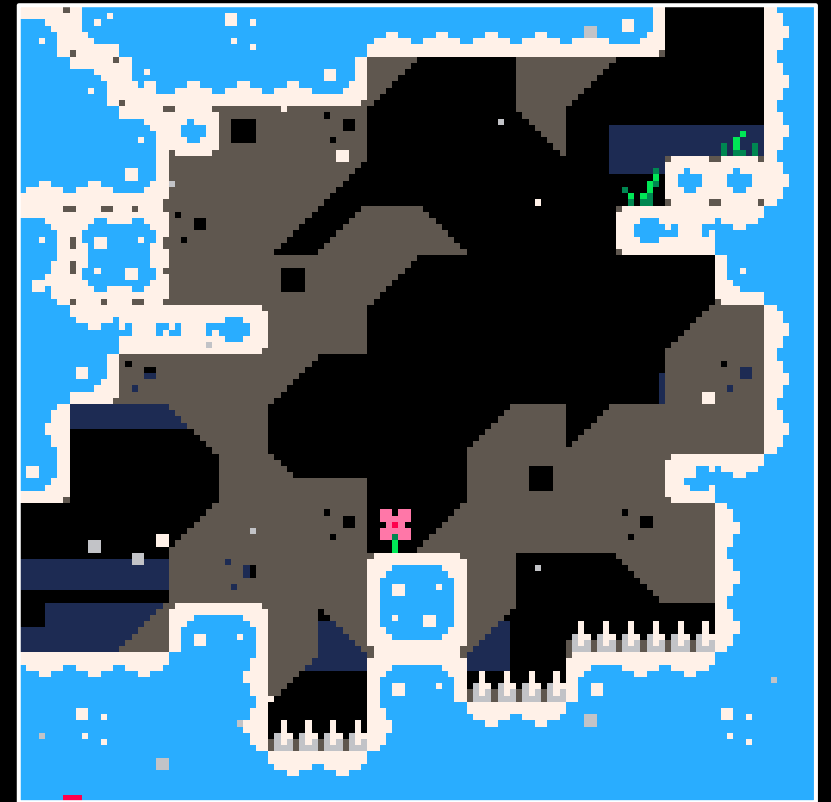
Akhil Madipalli · Maryam Malik · Jorge Manuel Torre



What is Celeste Classic?

- Small **precision platformer** built in PICO-8 (a fantasy console that mimics 8-bit consoles from the 1980s, programmed in Lua)
- Play as **Madeline** and climb a mountain, one screen at a time from bottom to top
- Controls: move left/right, **jump**, **dash** in 8 directions
Input is a **6-bit integer**: each bit is one button (left · right · up · down · jump · dash)
- Runs at **60 fps**; each frame the engine reads the input state and steps the physics
- **Level complete when Madeline exits off the top of the screen**
- Appears simple, but requires **frame-perfect jumps and dashes** to complete episodes

PICO-8 



Problem Statement

- **Goal:** Compare multiple models trained to complete the Celeste Classic platformer
- **Why Celeste Classic?**
 - Extremely precise controls, requires frame-perfect jumps & dashes
 - Sparse rewards, naive agents fail to converge if the only metric is to reach the goal state.
 - Expert data available via Tool-Assisted Speedrun (TAS) recordings
- **Core challenge:** sparse & delayed rewards in an environment precise control
- **Motivation:** Can TAS expert data accelerate/improve RL training? What matters more, algorithm or state representation?



Approach Overview

DQN (BASELINE & DUEL)

- Receives rewards for progress & penalties for failure
- Estimates which actions lead to the best long-term score
- Baseline DQN uses single NN to evaluate actions
- Dueling separates the NN into two: one rates the state, one rates the action

BEHAVIORAL CLONING (BC)

- Supervised learning by imitating expert playthroughs
- Trains from TAS demos
- Gaussian noise added to prevent overfitting
- Errors can compound when agent encounters an unknown situation
- Model can only perform as well as data

HYBRID

- Combines DQN and BC with TAS data
- Completes pre-training with BC by pre-loading replay buffer
- Fine-tune learning with DQN
- Primary hypothesis

CURRICULUM

- Divide tasks into smaller tasks to learn step-by-step
- Tasks become increasingly difficult
- Approach: Backwards curriculum over spawn position (change starting line each time)



Environment & State Space

■ Environment: Pyleste (Python PICO-8 emulator) in Gymnasium-compatible wrapper

■ State Vector – 87 Dimensions

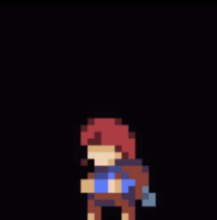
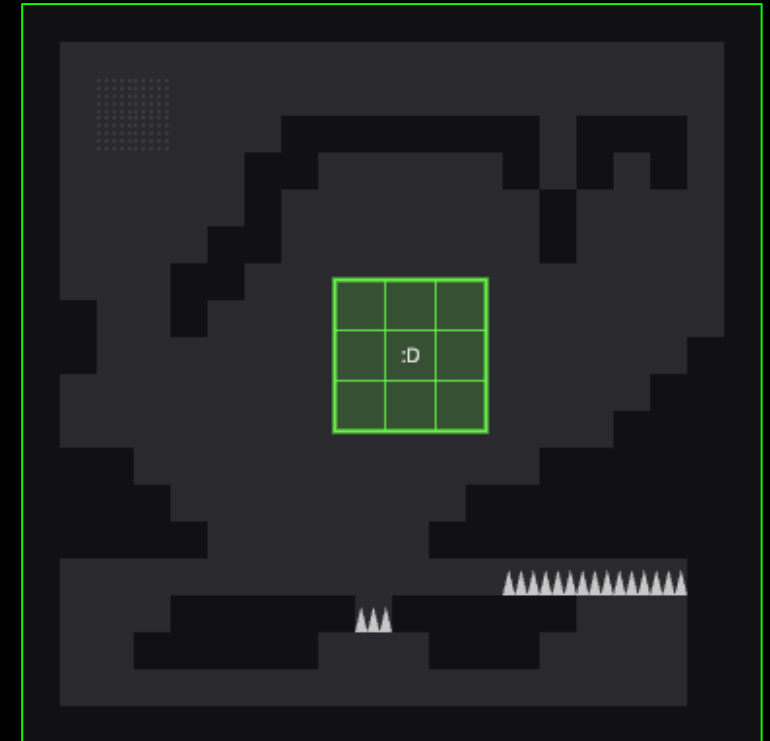
- ▶ 6 Scalar Features
 - x, y player position (normalized pixel coordinates)
 - spd_x, spd_y velocity
 - grace coyote-time frames (jump allowed after leaving platform)
 - djump dash availability (binary flag)

■ Semantic Tile Grid

- ▶ 9×9, centered on the player (81 values, not raw IDs)
- ▶ Encoding:
 - Out of bounds: -2
 - Spike: -1
 - Air: 0
 - Other (Decoration, fruit, non-deadly item): 0.5
 - Solid (Platform, wall): 1.0

■ Action Space: 15 discrete actions

- ▶ Idle, Left, Right, Jump ×3, Dash ×8 directions
- ▶ Reduced from 64 raw button combos (meaningless combos removed)



Network Architecture & Training Setup

Network

- **DQN:** Input(6) → Dense(256) → Dense(256) → Dense(128) → Output(15)
- **DuelingDQN:** shared trunk + separate V(s) and A(s,a) streams
- **Activation:** ReLU | **Optimizer:** Adam | **Grad clip:** norm ≤ 10

Reward Shaping

- +1.0 × height gained (main incentive)
- +0.5 explore new position | -0.1 stuck penalty
- -5 death | +500 level complete

Key Hyperparameters

Episodes	5,000
Max steps / episode	500
Learning rate	0.0005
Discount γ	0.99
ϵ start → end	1.0 → 0.05 (decay 0.999990)
Batch size	128
Replay buffer	500,000



Perception Fix

Before Raw Tile IDs

```
tile_grid.append(p8.tile_at(tx,ty))
```

Raw bytes 0–255 fed in.

Spike (ID=17) and decoration (ID=18) are numerically adjacent, thus the agent cannot differentiate between them.

After Semantic Encoding

5 classes and OOB sentinel:

- 2.0 out-of-bounds
- 1.0 spike (17,27,43,59)
- 0.0 air
- 0.5 other (decoration/fruit)
- 1.0 solid (landable)

Result Same DQN and HP

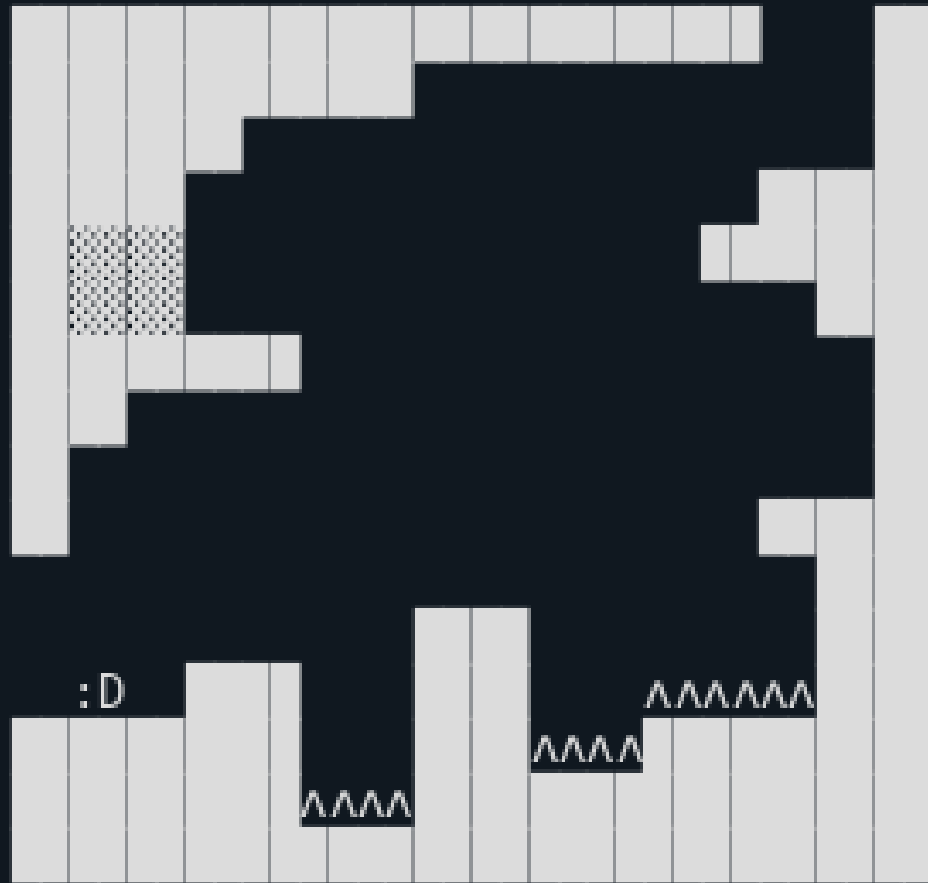
Raw IDs: 8% peak (collapse)
Semantic: 86% peak (sustained)

First completion:
Raw IDs: episode 2,600
Semantic: episode 1,500

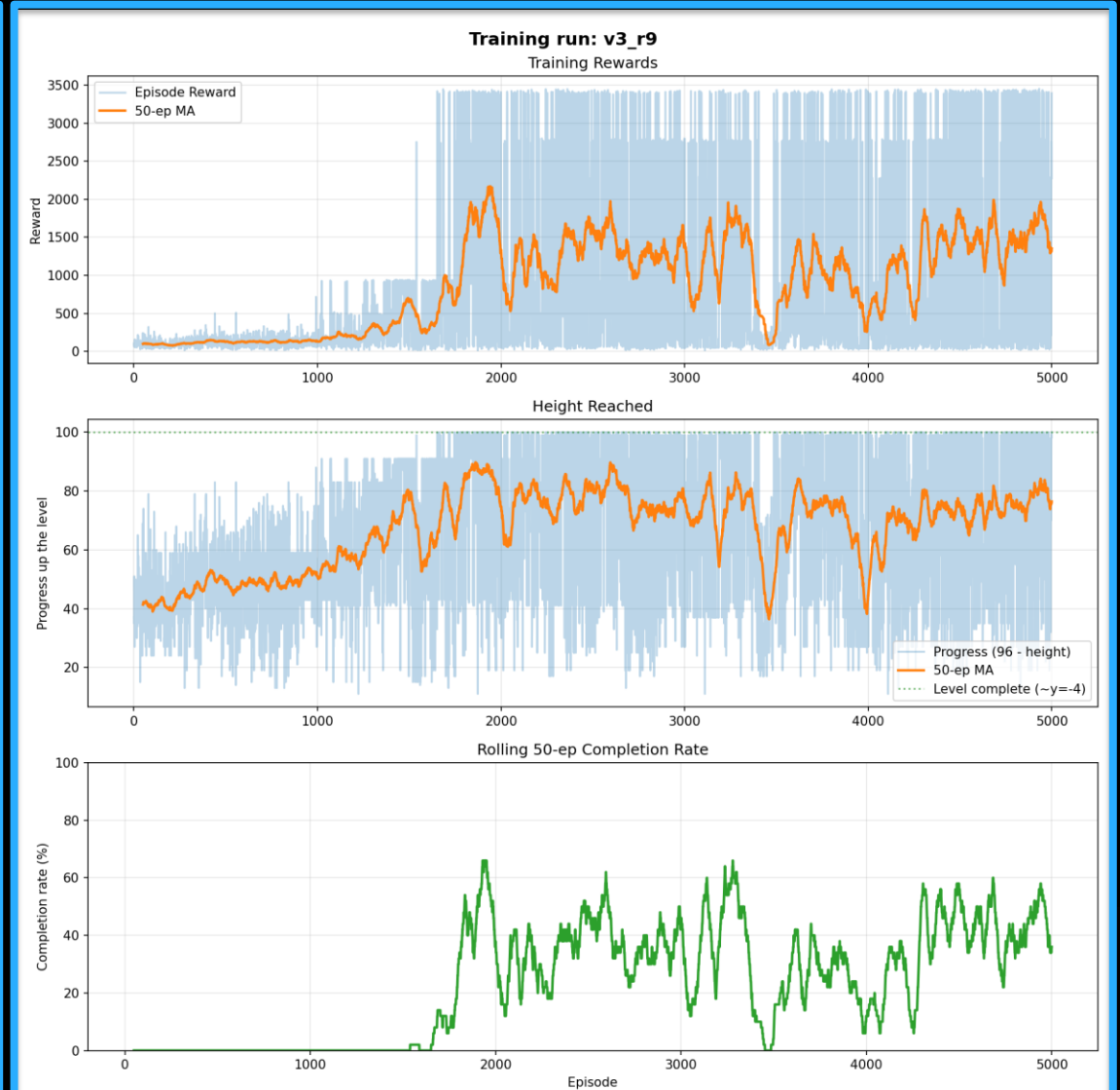
Takeaway: agent could see but couldn't interpret surroundings.



Training Run: v3_R9

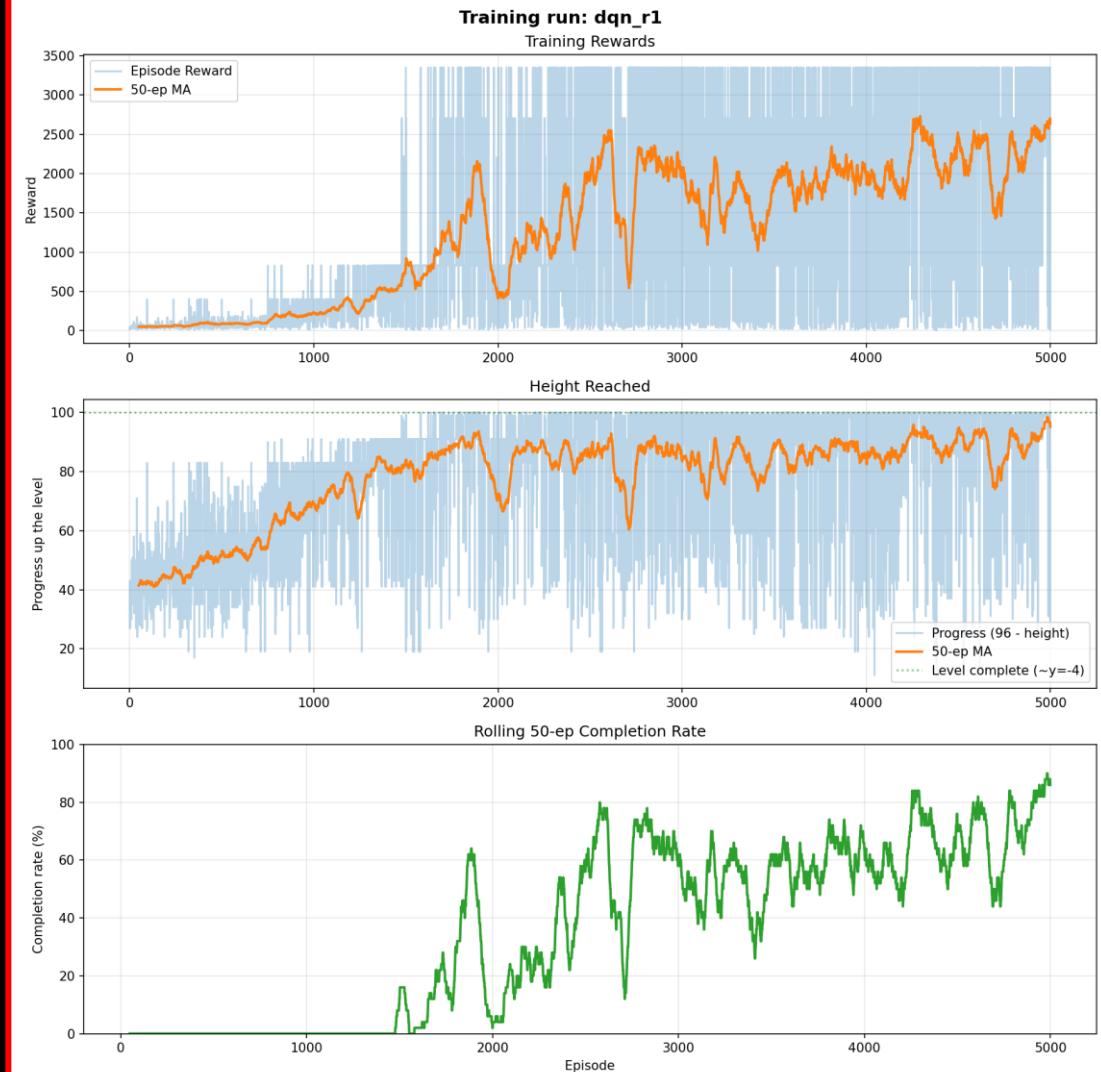


v3_r9 ep1 step 1 completions: 0



DQN Results - (dqn_r1)

- **Evaluation at $\epsilon=0.05$, 100 eps:** 68% completion
- **Training-time peak:** 86% (episode 1,950)
- **Lifetime training completions:** 1,783 / 5,000 (36%)
- **Median episode reward, roughly 2,700 (full path and terminal)**
- **Trajectory:** $y=13$ plateau -> first completions ep 1,500 -> stable 30 – 60% after
- **No catastrophic collapse** across 5,000 episodes
- **Reliably wall-jumps left wall, traverse upper section, clears $y < -4$ with a final wall jump and dash**



DQN Results - (dqn_r1)



Ablation Study

Results

- Plain DQN (dqn_r1): 68%
- Dueling + curiosity (v3_r9): 49%
- $\Delta = -19$ percentage points
- Plain DQN stayed stable
- v3_r9 collapsed mid-training

Issues with Complexity

■ Curiosity

Bonus rewards new (x,y) cells. After path is learned, it pulls the agent off-path.

■ Dueling

Few "redundant action" states -every frame demands precise input.

■ Conclusion

Architectural enhancements underperformed plain DQN with the same state encoding.



Behavioral Cloning Implementation

- **bc_r2** - supervised imitation of any% TAS, 200 epochs
 - 66 expert (state, action) transitions from a single TAS run
 - Action-weighted cross-entropy and Gaussian noise on kinematics
 - Train accuracy 70% · Val accuracy 67% (value set only 6 samples)
- **Evaluation at $\epsilon=0.05$, 100 eps:** 0% completion
- **Mean height (deterministic):** 55 (stuck mid-room loop)
- Cause of Failure - Covariate shift:
 - Sub-pixel drift puts the agent off the TAS distribution
 - Supervised loss has no recovery signal
 - Picks "closest seen" action, drifts further into a deterministic loop
- **Motivated the development of our Hybrid model (DQN + BC)**



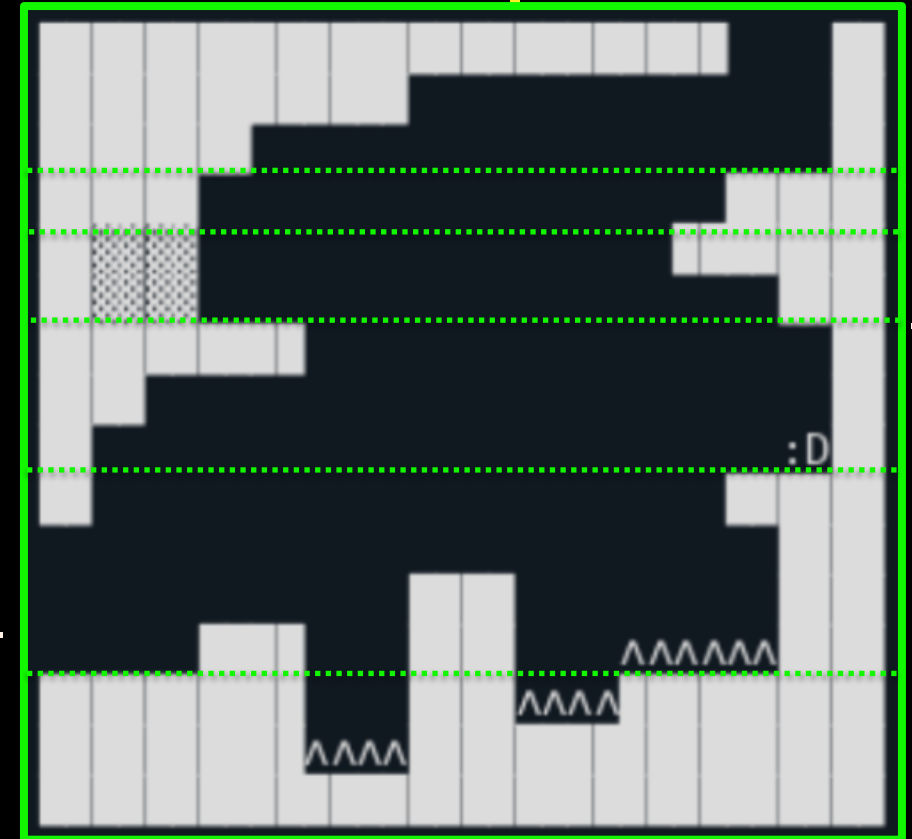
Hybrid (BC + DQN) Implementation

- **hybrid_r2** - DQN with TAS expert buffer (20% expert / 80% experience)
- **Evaluation at $\epsilon=0.05$, 100 eps:** 0% completion
- **Lifetime training:** 145 / 5,000 (2.9%)
- **Bug we found after testing:**
 - **build_expert_transitions** sets every TAS reward to zero
 - Q-learning bootstraps $Q(s,a) \approx r + \gamma \cdot \max Q(s',a')$
 - Reward = 0 on 20% of every batch
- **Honest framing:**
 - Implementation issue, not necessarily a property of imitation and reinforcement learning.
 - Correct version (replay TAS through environment, capture actual rewards) would possibly outperform plain DQN
 - No time to fix and rerun before deadline



Curriculum Learning Implementation

- **curriculum_r5** - 6-stage spawn curriculum, advance $\geq 50\%$ / 100 eps
 - ✓: Stage 1 (y=15), Stage 2 (y=30), Stage 3 (y=50)
 - x: Stage 4 (y=60), Stage 5 (y=71), Stage 6 (y=96)
- **Evaluation at $\epsilon=0.05$:** 0 / 100 completions
 - All deaths around h=72 (lower-mid bridge)
- **Why it failed:**
 - Each stage taught a local skill instead of the underlying task
 - Policy specialized to curriculum's intermediate spawn distributions
 - At y=96 (full level), did not encounter early-room states during the stages where it learned
- **Comparison: plain DQN trained from y=96** every episode, 68%



Final Comparison

DQN (WINNER - 68%)

- Trial-and-error RL with semantic perception
- 87-dimension state & 15 discrete actions
- Plain DQN (no Dueling) & Double DQN
- Reward shaping with milestone ramp

BEHAVIORAL CLONING (0%)

- Supervised imitation of TAS trajectory
- 66 expert (state, action) transitions
- Action-weighted CE + Gaussian noise
- Fails on covariate shift
- Evaluation: 0% (motivates Hybrid)

HYBRID (impl. bug 0%)

- DQN warm-started with TAS expert buffer (20% / 80%)
- Bug: TAS rewards stored as 0
- Anti-trains on optimal trajectory
- Honest framing: implementation, not method

CURRICULUM (0%)

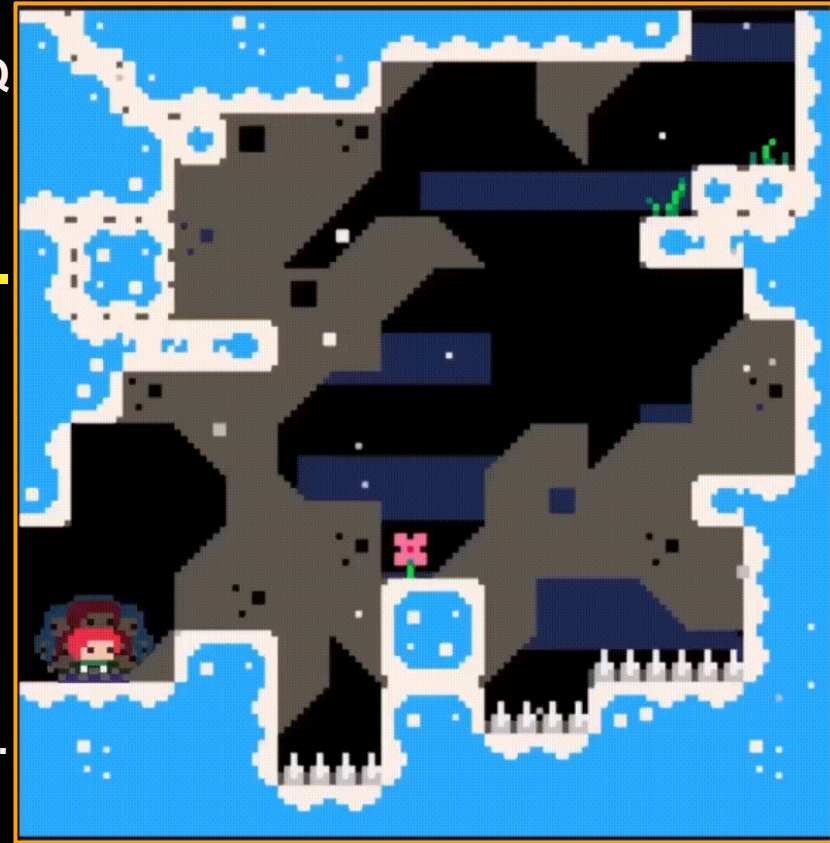
- 6-stage spawn curriculum: $y=15 \rightarrow 96$
- Advance when $\geq 50\%$ completion / 100 eps
- Stages 1–3 advanced; 4–6 timeout
- Specializes to curriculum's intermediate states
- Evaluation at 0% on full level



Related Work - Celeste-Bot (Genetic Algorithm)

- **effdotsh/Celeste-Bot** - C# / Unity recreation that completes the FULL game with a GA
- **Comparison:**
 - Learning signal: GA = selection on episode fitness | DQN = bootstrap Q
 - Exploration: GA = population diversity + mutation | DQN = ϵ -greedy
 - Memory: GA = none | DQN = FIFO replay buffer
 - Reward: GA = max height fitness | DQN = dense
- **Why GA works for full-game Celeste:**
 - Hundreds of nets per generation, one success is enough
 - Sparse reward is fine; no value function to bootstrap
 - No replay buffer to corrupt success signals
- **Caveat:** different impl (Unity, not Pyleste); no published training

Thus, this is proof the game is solvable rather than a numerical baseline.



Discussion - Tradeoffs & Next Steps

- Entire project trying to make DQN work
 - **Model Architecture:** tested Dueling DQN, then moved to Curriculum
 - **Perception and Rewards:** expanded the tile grid, added semantic labeling, scaled rewards for new heights
- Per-method tradeoffs (all on Celeste room 0, semantic state):
 - **Plain DQN:** stable, simple, 68% but ϵ -noise causes 24% spike deaths
 - **DuelingDQN:** faster early but pulls off optimal path and collapses
 - **BC:** no exploration needed but can't recover from any deviation (0%)
 - **Hybrid:** combines exploration & priors, needs careful reward handling
 - Curriculum: faster on intermediate stages but doesn't generalize back
- **Common pattern:** every "enhancement" matched or underperformed plain DQN. The dominant factor was state representation, not algorithm choice.
- **Next Steps:**
 - PPO + intrinsic curiosity (RND / ICM): on-policy, no FIFO buffer
 - NEAT or evolutionary strategies: matches what Celeste-Bot implemented
 - Larger expert dataset: 66 to 1,000+ would meaningfully aid BC accuracy



Website!



<https://jmtorr3.github.io/celeste-rl-web/>



Thank You!

Questions?

